Real-Time IP Camera Video Streaming System

Technical Engineering Verification & Project Assessment Report

Author: Independent Security Engineer **Category:** Computer Vision & Network Programming **Score Assessment:** 100 Marks Scheme

1. Executive Summary & System Architecture

This report details the implementation of a lightweight, standalone desktop IP camera streaming and surveillance application designed to eliminate proprietary vendor bloatware. Operating over local TCP/IP networks, the application connects to authorized cameras using open HTTP/MJPEG protocols. It processes raw video arrays, applies aspect-ratio-preserving transformations, injects contextual on-screen telemetry, handles asynchronous key events for still capture and containerized video recording, and implements defensive fault tolerance against transient network drops and abrupt terminations.

```
ip-camera-streamer/ ├── config.py # Environment variable parsing and strict URI scheme validation ├── stream_processor.py # Resizing, telemetry overlays, and exponential FPS smoothing ├── main.py # Main streaming loop, non-blocking I/O, and POSIX/Win32 signal traps ├── screenshots/ # Time-indexed still frames (ISO-8601 formatting) └── recordings/ # Finalized MP4 video containers (FourCC 'mp4v' encoding)
```

2. Verification & Acceptance Testing (TC01 – TC10)

The application was evaluated across ten deterministic test scenarios designed to validate nominal operations, extreme network failures, and abrupt lifecycle terminations.

ID	Test Scenario	Observed System Behavior	Status	Evidence
TC01	Live Stream Ingestion	Socket established; VideoCapture decoded frames at 30 FPS within 1.4s.	PASS	TC01_live.png
TC02	Malformed URI Scheme	config.py intercepted invalid protocol; exited code 1 with zero tracebacks.	PASS	TC02_err.png
TC03	Endpoint Unreachable	Requests probe timed out after 5 attempts; terminated cleanly with fatal log.	PASS	TC03_fail.png
TC04	Telemetry Overlays	Header bar rendered with alpha blend; timestamp and FPS updated per-frame.	PASS	TC04_ovl.png
TC05	Aspect Preservation	1080p source rescaled cleanly to 960×540 px without geometric distortion.	PASS	TC05_geom.png
TC06	Interactive Screenshot	Pressing 'S' wrote uncompressed image to disk with 2.0s UI confirmation.	PASS	TC06_cap.png
TC07	Active Video Recording	Pressing 'R' toggled MP4 recording; 10s clip generated and verified playable.	PASS	TC07_rec.png
TC08	Mid-Stream Wi-Fi Drop	Consecutive drops detected; reconnect sequence recovered on Wi-Fi restore.	PASS	TC08_drop.png
TC09	Esc Exit While Recording	Esc key triggered graceful shutdown; writer released; MP4 header finalized.	PASS	TC09_esc.png

ID	Test Scenario	Observed System Behavior	Status	Evidence
TC10	SIGINT (Ctrl+C) Interrupt	Signal caught; active writer closed before exit; zero 0 KB corruptions.	PASS	TC10_trap.png

3. Technical Questions: Part A — Python and Networking

A1. What is an IP address, and why must your computer and the camera be on the same local network?

An IP address is a logical numerical identifier assigned to a network interface facilitating packet routing via the Internet Protocol. The client workstation and camera must reside on the same local subnet (e.g., 192.168.1.0/24) because private IPv4 ranges defined under RFC 1918 are non-routable across the public Internet. Same-subnet deployment allows operating systems to map IP addresses to physical MAC addresses directly using Address Resolution Protocol (ARP) within a shared broadcast domain, bypassing complex Network Address Translation (NAT) or firewall traversal.

A2. Explain, step by step, what happens when opening http://192.168.1.23:8080/video. What part does HTTP play?

1. Addressing & ARP: The OS identifies the destination IP as local and issues an ARP broadcast to obtain the device's MAC address.
2. TCP Handshake: A standard 3-way handshake (SYN, SYN-ACK, ACK) completes on destination port 8080 to establish a reliable byte-stream channel.
3. HTTP Request: The client sends an HTTP GET request for resource path `/video`.
4. HTTP Role & Multipart Protocol: The camera acknowledges with `HTTP/1.1 200 OK` and specifies `Content-Type: multipart/x-mixed-replace; boundary=--boundarytag`. HTTP acts as the application layer transport that continuously serves discrete JPEG payloads separated by MIME boundary delimiters without closing the connection.

A3. What is a port number? Why does the URL include :8080, and what happens if it is omitted?

A port number is a 16-bit unsigned integer (0–65535) identifying a specific application or service on an operating system. Port 8080 is specified because mobile and user-space camera servers lack root privileges on Linux/Android required to bind to privileged ports below 1024. If omitted, HTTP clients default to port 80, resulting in a `Connection Refused (WinError 10061)` error as no listening daemon exists on that port.

A4. Difference between MJPEG over HTTP and RTSP? Which did your camera provide, and how was it verified?

MJPEG over HTTP is a sequence of standalone, intra-frame compressed JPEG images streamed via HTTP multipart boundaries. It lacks temporal inter-frame compression, resulting in higher bandwidth consumption but low decode latency and simple socket ingestion. RTSP (Real-Time Streaming Protocol, RFC 2326) is a stateful control protocol that negotiates media sessions using RTP packets carrying H.264/H.265 streams with inter-frame predictive compression (I, P, B frames). The camera provided an **MJPEG over HTTP** stream, verified via HTTP response headers yielding `Content-Type: multipart/x-mixed-replace` and discrete JPEG SOI marker bytes (`0xFFD8`).

A5. Why keep credentials out of source code? Show how this project secures them.

Hardcoding camera URIs, local network topologies, and passwords in source control creates severe vulnerabilities; public git repositories can be scanned by automated credential scrapers, granting unauthorized access to private surveillance streams. This project stores secrets in an untracked `.env` file:

```
CAMERA_STREAM_URL=http://192.168.1.128:8080/video CAMERA_NAME=Office_Entrance_Cam
```

Configuration is parsed at runtime via `python-dotenv` in `config.py`, while `.env` is explicitly ignored in `.gitignore`.

4. Technical Questions: Part B — Python Libraries

B1. What does cv2.VideoCapture do? What do isOpened() and read() return, and how does the code use them?

`cv2.VideoCapture` instantiates an image sequence reader binding OpenCV to system video devices, local files, or network streams. `isOpened()` returns a boolean confirming whether the backend initialized the stream pipeline; our code checks this to trigger backoff retries on failure. `read()` fetches, decodes, and returns a tuple `(ret, frame)` where `ret` is a boolean indicating frame retrieval success and `frame` is a NumPy ndarray. The code monitors `ret` to detect network disconnects and increment frame drop counters.

B2. A frame is a NumPy array. What does its shape tell you, and what is the shape of a 1280×720 colour frame?

The `shape` tuple denotes the multidimensional array structure ordered as `(Height, Width, Channels)`, corresponding to the matrix rows, matrix columns, and color plane depth. A 1280×720 color frame yields a shape of **(720, 1280, 3)**, indicating 720 horizontal scanlines, 1280 vertical pixel columns, and 3 interleaved 8-bit color channels.

B3. OpenCV stores colours in BGR order. Where does this matter, and what goes wrong if forgotten?

This affects custom pixel drawing (e.g., `cv2.putText`, `cv2.circle`, `cv2.rectangle`) and color space conversions. Supplying `(0, 0, 255)` draws pure red in BGR, but represents blue in standard RGB. If forgotten when saving via external libraries or plotting with Matplotlib, channel ordering causes severe chromatic distortion: red recording indicators appear deep blue, and human skin displays with an uncoordinated blue/green hue.

B4. What is cv2.waitKey() for? Why does the window freeze without it? How are Esc, S, and R detected?

`cv2.waitKey(delay)` performs two operations: it pauses execution for specified milliseconds to listen for keyboard events, and it pumps the native operating system HighGUI event loop to handle paint, redraw, and focus events. Without it, the OS treats the window as unresponsive and halts visual updates. Keycodes are extracted by masking the lowest 8 bits: `key = cv2.waitKey(1) & 0xFF`. Esc corresponds to integer 27, S to `ord('s')` / `ord('S')`, and R to `ord('r')` / `ord('R')`.

B5. What was Requests used for? Why check the camera before handing the URL to OpenCV?

The Requests library executes an upfront, non-blocking HTTP GET preflight probe with an explicit 5-second socket timeout (`timeout=5, stream=True`). This is necessary because `cv2.VideoCapture` relies on internal FFmpeg network drivers that lack short default connection timeouts; pointing OpenCV directly at an unresponsive IP or filtered port can cause the execution thread to freeze for up to 60 seconds without feedback.

5. Technical Questions: Part C — Video Processing

C1. How is FPS estimated? Why does it fluctuate, and how is it stabilized?

Instantaneous FPS is derived by inverting the measured elapsed time between consecutive frame decodes: `1.0 / (t_now - t_prev)` using `time.perf_counter()`. Values fluctuate due to Wi-Fi packet jitter, OS thread scheduling, and variable JPEG compression complexity. The readout is stabilized using an Exponential Moving Average (EMA):
$FPS_{smoothed} = \alpha \times FPS_{smoothed} + (1 - \alpha) \times FPS_{instantaneous}$
with a smoothing factor of $\alpha = 0.9$, dampening erratic visual jumps.

C2. How was aspect-preserving resizing implemented? Show calculations for 1920×1080 to width 960.

The scale ratio is computed by dividing the target display width by the source frame width, then multiplying the source height by this ratio:
`Scale Ratio = 960 / 1920 = 0.5`
`Target Height = int(1080 × 0.5) = 540`
The frame is processed using `cv2.resize(frame, (960, 540), interpolation=cv2.INTER_LINEAR)`, preserving the 16:9 aspect ratio without vertical compression or horizontal stretching.

C3. What does cv2.VideoWriter require? What happens if written frames differ in size from declared?

`cv2.VideoWriter` requires the destination filepath, a 4-character FourCC codec identifier (e.g., `cv2.VideoWriter_fourcc(*'mp4v')`), target FPS, and declared frame geometry as a **(Width, Height)** tuple (the inverse of NumPy's row/column ordering). If frame dimensions passed to `writer.write()` do not match the declared tuple, OpenCV silently discards the incoming frame arrays, producing an empty, corrupted 0 KB video file.

C4. Why do overlays and encoding add performance cost? State two ways to reduce this overhead.

Overlays require memory copying and in-place pixel rasterization across high-resolution matrices. Disk recording demands continuous color conversion, spatial compression, and disk I/O synchronization on every frame cycle. Overhead can be reduced by:
1. Threaded Pipeline Decoupling: Isolating frame decoding and encoding into separate background worker threads using thread-safe FIFO queues (Producer-Consumer architecture).
2. Overlay Caching & Subsampling: Recomputing text overlays at reduced intervals (e.g., 5 Hz) rather than recalculating font metrics on every single 30 FPS video frame.

C5. How are screenshot and recording filenames formatted to prevent collisions?

Outputs are formatted using ISO-8601-derived chronological timestamps down to the microsecond level:

```
screenshot_path = f"screenshots/screenshot_{datetime.now().strftime('%Y%m%d_%H%M%S_%f')}.jpg"
recording_path = f"recordings/recording_{datetime.now().strftime('%Y%m%d_%H%M%S')}.mp4"
```

Because system clocks advance monotonically, every generated artifact receives a unique filename, eliminating accidental overwrite risks during rapid successive user keypresses.

6. Technical Questions: Part D — Critical Thinking

D1. What occurs if Wi-Fi drops for 10s mid-stream? Explain reconnect logic and threshold choices.

When Wi-Fi drops, `cap.read()` returns `ret=False`. The application tracks consecutive frame drops; after 30 failed reads (~1.0 second of silence), it flags a connection drop. It releases the stale capture handle, pauses 1.0s to allow the OS to clear TCP socket state, and begins a retry loop: up to `MAX_RECONNECT_ATTEMPTS=5` spaced by `RECONNECT_DELAY_SEC=2`. This 12-second recovery window accommodates transient Wi-Fi drops while preventing the application from running as a zombie process if the camera is permanently shut down.

D2. Why must VideoCapture and VideoWriter be released on every exit? What happens to unreleased recordings?

Releasing handles returns system camera resources and socket descriptors to the operating system. For `cv2.VideoWriter`, calling `release()` is critical because containerized video formats (such as MP4) cache index headers and container metadata (the `moov` atom) in memory during streaming. If the process terminates without closing the writer, this metadata is never flushed to disk, leaving the media container unindexed and unplayable by standard media decoders.

D3. What privacy and ethical responsibilities accompany video streaming? How does this design address them?

Surveillance software creates risks of non-consensual biometric monitoring, privacy intrusion, and unauthorized data leakage. This application addresses these concerns through several safeguards: it operates exclusively over local network interfaces without cloud telemetry relay; credentials and streaming paths remain isolated from git tracking; a prominent visual indicator (`● REC [MM:SS]`) alerts users when recording is active to prevent covert capture; and output media folders are excluded from version control to prevent local surveillance data from being exposed publicly.

D4. If the viewer had to display ten cameras simultaneously, what fails first and how would you redesign it?

Primary Failure Point: A single-threaded blocking loop would freeze immediately. Decoding 10 MJPEG feeds sequentially on one thread causes aggregate frame rates to collapse to 1–3 FPS, as each network I/O read blocks the shared thread, creating compounding frame latency across all streams.
Redesign: Implement a multi-threaded architecture with a dedicated worker thread per camera. Each worker fetches frames into a single-slot buffer (`deque(maxlen=1)`) to drop stale frames. The main GUI thread reads the latest frame from each buffer, composites them into a 2×5 tiled NumPy matrix, and renders the unified canvas in a single draw pass.

D5. What was the most difficult bug encountered? How was it resolved, and what would you do differently?

The Defect: `cv2.VideoWriter` generated corrupted, 0 KB unplayable video files despite valid video playback in the GUI window.
Root Cause: A dimension order mismatch. NumPy arrays report dimensions as `(Height, Width, Channels)`, whereas the `cv2.VideoWriter` constructor expects `(Width, Height)`. Inverting the tuple caused OpenCV to silently drop incoming frames.
Resolution: Explicitly mapped dimensions as `(frame.shape[1], frame.shape[0])` and added dimension assertions before initializing the writer. In future implementations, using structured dataclasses for frame telemetry will help prevent coordinate order bugs.